

流水线与指令级并行

多条指令怎样合法重叠而不破坏顺序语义

408 · 计算机组成原理 Topic CO-04

母问题：怎样重叠执行，同时保持与 ISA 顺序轨迹等价？

CO-03 已给出一条指令内部的值依赖和提交边界。CO-04 把多条指令放到同一时间轴上，生成链是

Instruction Dependencies → Stage / Resource Occupancy → Need-Ready Constraints → Legal Overlap → For

箭头表示约束求解顺序：先承认程序依赖和资源事实，再判断哪些重叠合法；不是先背“停几拍”，再倒推理由。

本册定位与 Owner 边界

- **Owns:** stage timing、latency/throughput、流水寄存器、结构/数据/控制 hazard、need/ready、forwarding、stall、flush、CPI、精确异常与 408 范围内 ILP 扩展。
- **Uses:** CO-02 的 ISA 顺序语义；CO-03 的单指令值依赖、资源和提交；CO-06/07 的 miss/fault 延迟接口。
- **Stops:** 不重讲单指令微操作；不拥有 Cache/TLB 状态机；多核一致性、完整乱序执行和编译器后端只作 Extension。

目录

1	流水化从哪里生成	3
1.1	朴素方案与失败	3
1.2	吞吐率与延迟	3
2	时间模型：Produced、Ready、Need 与 Commit	3
3	Stage 与时空图：先声明模型	3
4	三类 Hazard：三种约束冲突	4
4.1	结构冲突	4
4.2	数据冲突	4
4.3	控制冲突	4
5	三种控制动作：Forward、Stall、Flush	5

6 贯穿母例：LOAD 后立即 ADD	5
7 控制冒险与预测状态机	5
7.1 惩罚从决策位置生成	5
7.2 两位饱和计数器	6
8 CPI 与总时间：从时空图计数	6
9 精确异常与提交顺序	6
10 代价、边界与 ILP 扩展	6
11 Flynn 与多处理器：并行对象先分清	7
12 五列概念边界	7
13 做题控制协议	7
14 全科坐标与跨册交接	8
15 考纲映射与一页压缩	8
16 来源处理	9

1 流水化从哪里生成

1.1 朴素方案与失败

若一条指令依次经历多个功能阶段，非流水实现会让当前未使用的部件闲置。把长组合路径切成 k 段，并在段间加入流水寄存器后，不同指令可以占用不同阶段。

设各段组合延迟为 T_i ，流水寄存器开销为 T_{reg} ，则

$$T_{clk} \geq \max_i T_i + T_{reg}.$$

理想情况下 n 条指令总时间为

$$T_{pipe} = (k + n - 1)T_{clk},$$

但这只是无 hazard、无 miss、段延迟均衡的成本模型。

1.2 吞吐率与延迟

流水线主要提高稳态吞吐率，不保证降低单条指令 latency。更深的流水可能缩短 T_{clk} ，却增加流水寄存器开销、填充/排空、分支惩罚和旁路复杂度。因此比较性能必须回到

$$T_{CPU} = IC \times CPI \times T_{clk}.$$

2 时间模型：Produced、Ready、Need 与 Commit

对一个值 v ，必须至少标四个时刻：

- **Produced**: 功能部件已经计算出 v ；
- **Ready**: 从某条实际路径看，消费者能够获得 v ；
- **Need**: 消费者所在阶段必须使用 v ；
- **Commit**: v 或其他效果写入软件可见架构状态。

RAW 依赖是否需要停顿取决于

$$t_{ready}(producer, path) \leq t_{need}(consumer).$$

若不等式失败，必须改变时间 (stall) 或改变路径 (增加 forwarding)；若值尚未 produced，任何旁路都无能为力。

Ready 不等于 Commit

forwarding 可以让 ALU 结果在写回前被下一条指令使用，但寄存器堆此时可能仍是旧值。可用时刻回答“谁能拿到”，提交时刻回答“软件可见状态何时改变”。

3 Stage 与时空图：先声明模型

经典五级教学模型常写为 IF、ID、EX、MEM、WB，但停顿数依赖以下假设：

- 各 stage 的真实职责与输入/输出；
- 寄存器堆同周期写/读的先后关系；
- 指令存储与数据存储是否共享端口；
- 分支在哪一级比较并产生目标；
- forwarding 的起点和终点；
- memory 是否固定一拍返回。

周期	IF	ID	EX	MEM	WB
1	I_1				
2	I_2	I_1			
3	I_3	I_2	I_1		
4	I_4	I_3	I_2	I_1	
5	I_5	I_4	I_3	I_2	I_1

时空图不是装饰：每个格子同时表示 stage、资源占用和流水寄存器边界。发生 stall 时必须说明哪些前级保持、哪个控制包被清零为 bubble、哪些后级继续推进。

4 三类 Hazard：三种约束冲突

4.1 结构冲突

同一周期两项工作需要同一不可并用资源，例如 IF 与 MEM 共用单端口存储器。解决路径只有三类：复制/增加端口，以空间换吞吐；错开调度，以周期换资源；或改变组织。是否冲突由题设资源图决定，不能由“无数据依赖”推出无冲突。

4.2 数据冲突

- **RAW**：消费者读生产者将写的新值，是真依赖；
- **WAR**：较年轻指令写得过早，破坏较老指令尚未完成的读；
- **WAW**：两个写同一目标的提交次序颠倒。

在单发射、按序发射、按序读写的简单五级流水中，通常只有 RAW 形成实际 hazard；WAR/WAW 需要乱序读写、多发射或不同完成延迟才可能出现。这个结论必须绑定模型，不能写成所有流水线的定律。

4.3 控制冲突

分支方向/目标尚未确定时，前级已经取入后续指令。等待、静态/动态预测、延迟槽和尽早决策都在交换平均停顿、硬件/编译器复杂度和错误路径成本。无论策略如何，错误路径效果不能提交。

5 三种控制动作：Forward、Stall、Flush

动作	保留/推进什么	阻止什么	使用条件
Forward	正常推进并改用旁路值	消费旧值	值已经 produced，且实际路径可达 need 端
Stall	保持 PC/前级状态或局部队列	未准备好的消费者前进	need 早于所有 ready 路径，或资源冲突
Flush	保留正确较老状态	错路/异常后的年轻效果提交	分支误判、异常、重定向

forwarding 不是“打开一个总旁路”。ALU → EX、load data → EX、result → branch compare、store data → MEM 是不同路径；题目没给路径，就不能假设存在。

6 贯穿母例：LOAD 后立即 ADD

$I_1 : \text{LOAD } R1, 0(R2), \quad I_2 : \text{ADD } R3, R1, R4.$

在经典五级模型中： I_1 的 EA 在 EX 产生，数据通常在 MEM 末尾产生； I_2 在自己的 EX 开始需要 $R1$ 。若相邻发射，数据的 ready 晚于 need，必须至少让消费者后移，直到存在可达旁路。

	1	2	3	4	5	6	7
I_1	IF	ID	EX	MEM	WB		
I_2		IF	ID	stall	EX	MEM	WB

该“一拍”只在上述 stage、memory latency、寄存器时序和 MEM-to-EX 旁路假设下成立。若分级更深、memory 多周期或旁路端点不同，必须重新比较 need/ready。

母例故意暴露的错误路径

“有 forwarding，所以永不停顿”忽略了值尚未产生；“LOAD 在 EX 算出了结果”把 EA 当成 load data；“ $I1$ 已经转发，所以已经完成”把 ready 当 commit。

7 控制冒险与预测状态机

7.1 惩罚从决策位置生成

若分支在第 d 级末尾才确定，错误路径已经进入的年轻指令数量由前端组织和预测时机决定。penalty 不是“分支固定几拍”，必须从决策级、已推进阶段和 flush 范围数出。

平均 CPI 贡献可写为

$$CPI_{branch} = f_{branch} r_{mispredict} P_{mispredict},$$

其中 $P_{mispredict}$ 必须是额外周期；若题目给的是总分支周期，需先剥离理想基线，避免重复计数。

7.2 两位饱和计数器

两位预测器是四状态 FSM，单次相反结果只使状态向另一方向移动一步。题目必须给初态、实际 T/NT 序列和采用高位/状态预测规则；循环“通常只错一次”不是无条件定律，初态和连续多轮会改变结果。

8 CPI 与总时间：从时空图计数

实际 CPI 可以分解为

$$CPI_{actual} = CPI_{ideal} + CPI_{struct} + CPI_{data} + CPI_{control} + CPI_{memory} + \cdots$$

分项只有在互斥或题设明确时才能直接相加；若一个 memory miss 同时阻止了后续依赖，不能把同一停顿重复归入两项。

有限指令序列的总周期应从时空图第一拍数到最后一条的规定完成/提交点。填充与排空已经包含在图中，不应再机械追加 $k - 1$ 。

9 精确异常与提交顺序

精确状态要求：异常指令之前的较老指令效果已经成立；异常指令和更年轻指令的禁止副作用没有提交。顺序流水可以冻结/清零写使能并 flush；更复杂实现把 produced/finished 与 ordered commit 分开。

execution may overlap $\wedge$ architectural trace remains recoverable

这条不变量连接 CO-03 的单指令 Write Set、CO-04 的多指令年龄顺序和 X-B01 的异常入口。OS handler 不在本册展开。

10 代价、边界与 ILP 扩展

机制	换来的能力	新成本	最小失败条件
更深流水	更短候选时钟、更多重叠	寄存器开销、分支/依赖周期数增加	最慢段/寄存器开销不降
Forwarding	减少已产生值的 RAW stall	比较器、MUX、布线和时序压力	值尚未产生或端点不可达
分支预测	降低平均控制停顿	预测状态、能耗、错误 flush	低可预测性 or 高 penalty
多发射	峰值 IPC 大于 1	端口、依赖检查、带宽	指令不独立或资源不足
VLIW	编译期显式并行、简化部分硬件	代码密度、调度和兼容性压力	动态延迟/并行不足

超标量、乱序、寄存器重命名、ROB、VLIW 和多核属于“同一依赖/资源/提交框架怎样扩张”的 Extension。408 主干只保留能解释 WAR/WAW、动态调度和 ordered commit 的最小接口，不把现代实现参数写成固定规律。

11 Flynn 与多处理器：并行对象先分清

流水线和超标量主要是在一条指令流内重叠；多处理器则增加执行上下文或指令流。按指令流/数据流分类：

类别	指令流/数据流	典型结构	题面判据
SISD	单/单	单核顺序处理器	一条指令处理一个数据序列
SIMD	单/多	向量机、向量指令	同一控制对多个数据元素执行
MISD	多/单	少见的专用容错结构	多条指令作用于同一数据流
MIMD	多/多	多核、SMP、通用多处理器	不同核心可执行不同线程/程序

向量处理属于 SIMD 的数据级并行；多核/SMP 属于 MIMD 的共享或分布式执行资源，不能把“多发射每拍多条”直接说成“多核”。硬件多线程则是在一个核心内保存多个线程上下文，在某线程等待或资源空闲时切换发射；它提高资源利用率，不自动增加物理 ALU 数量。

多处理器题仍沿本册三条约束检查：每个执行流的依赖与 ready/need、共享 Cache/总线/内存的资源冲突、最终架构提交是否符合各自 ISA 顺序。题目若进一步问 Cache 一致性、锁或线程调度，应交给相应跨册/OS Owner；本节只确定并行层次与硬件接口。

并行层级的停止条件

先回答“增加的是数据元素、同一指令流中的发射宽度、线程上下文，还是物理核心/指令流”；再检查共享资源与提交边界。若无法指出新增并行对象，就不能仅凭“同时”判定属于 SIMD、超标量或多核。

12 五列概念边界

概念 A	≠ 概念 B	真正区别与题目信号	混淆后果
Latency	≠ Throughput	单条穿越时间 vs 单位时间完成数	误称流水必降单条延迟
依赖	≠ Hazard	程序语义关系 vs 当前时间/资源下的冲突	把所有 RAW 都算固定停顿
Produced	≠ Ready	已算出 vs 通过实际路径可被消费者拿到	假设不存在的旁路
Ready	≠ Commit	消费者可用 vs 架构状态已更新	精确异常和寄存器值判断错误
Stall	≠ Flush	等待后继续同一路径 vs 作废错误/年轻状态	保留了不应提交的控制包
ILP	≠ 多核并行	单指令流内部重叠 vs 多核心/任务并行	把 OS 调度和 Cache 一致性塞入流水线题

13 做题控制协议

1. 写 stage 定义、每级资源、寄存器读写时序、memory latency 和已有 forwarding；
2. 逐条列源/目的寄存器和控制转移，标 producer、consumer；

- 3. 对每个相关值标 produced、ready、need、commit;
- 4. 先画无冲突时空图，再叠加结构、数据和控制约束;
- 5. forwarding 必须写起点、终点和到达时刻; 剩余 ready-need 差才用 stall;
- 6. 分支写预测状态、决策级和 flush 范围;
- 7. 从最终时空图统计总周期/CPI，不重复添加填充或同一停顿;
- 8. 用年龄顺序和 Write Set 检查错误/异常路径没有提交。

压缩成两问

流水线题先问：这个值沿现有路径什么时候 ready，消费者什么时候 need？这个副作用在哪个年龄/时钟边界才允许 commit？

14 全科坐标与跨册交接

Map Coordinate: Timing / Availability

CO-04 接收 CO-03 的单指令路径，给每个值标出 produced、ready、consumer need 和 architectural commit；它向 CO-I01 输出合法重叠、forward/stall/flush 与精确提交顺序。阶段名称不能替代这些时刻。

吞吐量是稳定填满后的单位时间完成量, latency 是单条指令从进入到结果/提交的时间; 两者和 clock rate、CPI、分支 penalty、miss stall 一起才形成 Cost。流水线加深或并行度增加不自动降低完整 CPU time。

15 考纲映射与一页压缩

传统考点	本册机制位置	下游接口
流水线性能	stage 延迟、填充/排空、CPU time	Rules 性能工具箱
结构/数据/控制冒险	资源、need/ready、Next PC	CO-03/CO-B01
转发、停顿、冲刷	三种控制动作与接口条件	具体时空图题
分支预测	预测 FSM、决策级、mispredict penalty	CO-I01
超标量/乱序/VLIW	ILP Extension 与按序提交边界	不扩张为多核 Topic

一句话

流水线把多条指令放到同一时间轴上：先用 need/ready 和资源约束判断哪些重叠合法，再用 forward、stall、flush 修复冲突，最终只允许符合年龄顺序和 ISA 的效果提交。

自测：为什么 forwarding 不能消灭所有 RAW？为什么 WAR/WAW 在简单五级顺序流水中通常不形成 hazard？为什么更深流水不保证更快？为什么分支 penalty 必须从决策级和 flush 范围数出？

16 来源处理

本册吸收归档《指令流水线》和原 Topic README 的 stage、hazard、forwarding、分支与性能主干。固定五级停顿数、特定寄存器半拍规则、现代 CPU 深度/宽度和“所有 RISC/CISC 性能相当”等实现结论均未升级为稳定定律；Flynn、多核、完整乱序和编译器后端只保留 Extension 指针。